

Spark2 SDK: Repository Structure and API

- Language: Python 3.8+
- Backend: C++ SDK exposed via `_spark2_sdk_cpp` (pybind11)

Repository Structure

The installable Python package lives under `spark2_python_dist/` in the Spark2 SDK repository. Shared robot configuration is bundled at the repository root in `configuration/`.

```
spark2_sdk/
|-- configuration/                (robot configuration)
|-- spark2_python_dist/
|   |-- pyproject.toml
|   |-- spark2_sdk_python.md
|   |-- spark2_sdk/
|       |-- __init__.py
|       |-- spark2.py             (main SDK class API)
|       |-- configurator.py       (configuration API)
|       |-- kinematics.py         (kinematics API)
|       |-- types.py              (shared data types/enums)
|       |-- convertor.py          (internal C++/Python type conversion)
|       |-- lib/                  (pre-built shared library)
|   |-- examples/
|       |-- utils.py              (example helpers)
|       |-- move_point.py
|       |-- move_path.py
|       |-- teach_and_playback.py
|       |-- set_configuration.py
|       |-- calculate_kinematics.py
```

Platform and toolchain subdirectories under `lib/` are selected automatically at import time based on OS, CPU architecture, and (on Windows) the MSVC runtime version.

Core SDK API (`Spark2`)

API	Description	Group
<code>Spark2(config_prefix_path="")</code>	Create SDK client; optional path prefix for configuration files.	constructor
<code>start()</code> , <code>stop()</code>	Start or stop arm runtime session.	connection
<code>enable_arm_joint(arm_state)</code>	Enable arm joints by 6-element boolean state (<code>JointState6b</code>).	connection
<code>set_arm_smoothing_method(method)</code>	Select smoothing strategy for single-target arm motion.	motion mode

API	Description	Group
<code>move_pos(arm_pos, v=50, t=0.0)</code>	Command joint-space target position.	manual motion
<code>move_pos_path(path, v_path, t_path)</code>	Command sequence of joint-space waypoints.	manual motion
<code>move_vel(arm_vel, a=50, t=0.0)</code>	Command joint-space velocity.	manual motion
<code>move_ee_point(pose, v=50, t=0.0)</code>	Move end-effector to cartesian pose (point mode).	manual motion
<code>move_ee_point_path(path, v_path, t_path)</code>	Move through end-effector pose waypoints (point mode).	manual motion
<code>move_ee_line(pose, v=50, t=0.0)</code>	Move end-effector to cartesian pose (line mode).	manual motion
<code>move_ee_line_path(path, v_path, t_path)</code>	Move through end-effector pose waypoints (line mode).	manual motion
<code>move_tool_point(pose, v=50, t=0.0)</code>	Move tool frame to cartesian pose (point mode).	manual motion
<code>move_tool_point_path(path, v_path, t_path)</code>	Move tool frame through cartesian waypoints (point mode).	manual motion
<code>move_tool_line(pose, v=50, t=0.0)</code>	Move tool frame to cartesian pose (line mode).	manual motion
<code>move_tool_line_path(path, v_path, t_path)</code>	Move tool frame through cartesian waypoints (line mode).	manual motion
<code>go_home(v=50, t=0.0)</code>	Move robot arm to predefined home pose.	manual motion
<code>move_gripper_pos(pos, v=50, t=0.0)</code>	Command gripper joint position.	manual motion
<code>start_teach(), stop_teach()</code>	Enter or exit teach mode.	teach mode
<code>start_playback(), stop_playback(), reset_playback()</code>	Control playback mode lifecycle.	playback mode
<code>get_pos(), get_vel(), get_tor()</code>	Read current joint position, velocity, and torque.	feedback
<code>get_ee_pose(), get_tool_pose()</code>	Read current end-effector/tool cartesian pose.	feedback
<code>is_arm_joint_enabled()</code>	Read per-joint arm enable state.	status

API	Description	Group
<code>get_status()</code> , <code>print_status(status)</code>	Read or print aggregated runtime status (<code>SystemStatus</code>).	status
<code>get_configurator()</code> , <code>get_kinematics()</code>	Access specialized subsystems for config/kinematics.	subsystems

Configuration API (`Configurator`)

API	Description
<code>set_tool_offset(offset)</code> , <code>get_tool_offset()</code>	Set or get tool frame offset from end-effector frame.
<code>set_arm_pos_limits(limit)</code> , <code>set_arm_vel_limits(limit)</code>	Set joint position/velocity safety limits.
<code>get_arm_pos_limits()</code> , <code>get_arm_vel_limits()</code>	Get active joint safety limits.
<code>get_arm_max_pos()</code> , <code>get_arm_max_vel()</code>	Get hardware max position/velocity constraints.
<code>get_arm_max_pos_jump()</code> , <code>get_arm_max_vel_jump()</code>	Get anti-jump protection limits.
<code>get_arm_max_pos_follow_error()</code> , <code>get_arm_max_vel_follow_error()</code> , <code>get_arm_max_tor_follow_error()</code>	Get following-error thresholds.
<code>set_gripper_pos_limits(limit)</code> , <code>get_gripper_pos_limits()</code>	Set or get gripper position limits.
<code>get_gripper_max_pos()</code>	Get gripper hardware max range.

Kinematics API (`Kinematics`)

API	Description
<code>forward_kinematics(arm_pos)</code>	Compute cartesian pose from 6-joint arm state.
<code>inverse_kinematics(ee_pose, arm_pos)</code>	Solve 6-joint arm state for a target cartesian pose; returns (<code>bool</code> , <code>JointState6f</code>).
<code>euler_to_quaternion(euler)</code>	Convert intrinsic XYZ Euler angles to quaternion.
<code>quaternion_to_euler(quat)</code>	Convert quaternion to intrinsic XYZ Euler angles.
<code>ee_to_tool_pose(ee_pose, tool_offset)</code>	Transform end-effector pose into tool pose.
<code>tool_to_ee_pose(tool_pose, tool_offset)</code>	Transform tool pose into end-effector pose.

Types (`types.py`)

Type / Enum	Description
<code>Position</code> , <code>Quaternion</code> , <code>EulerAngle</code>	Cartesian position, orientation, and intrinsic XYZ Euler angles.
<code>Pose</code>	Cartesian pose = <code>Position</code> + <code>Quaternion</code> orientation.
<code>JointState6f</code> / <code>JointState1f</code>	6-DOF arm joint values / 1-DOF gripper value.
<code>JointState6b</code> / <code>JointState1b</code>	6-DOF arm enable flags / gripper enable flag.
<code>JointState6u</code> / <code>JointState1u</code>	Per-joint diagnostic bitmasks for arm and gripper.
<code>List[T]</code>	Waypoint sequence (e.g. <code>List[Pose]</code> , <code>List[JointState6f]</code>).
<code>RobotJointStatef</code> / <code>RobotJointStateb</code>	Combined arm+gripper state in float or bool forms.
<code>JointLimits6f</code> / <code>JointLimits1f</code>	Per-joint min/max limits for arm and gripper (<code>LimitPair</code> per joint).
<code>SmoothingMethod</code>	<code>LINEAR</code> , <code>COS</code> , <code>CUBIC</code> , <code>QUINTIC</code> , <code>NONE</code> .
<code>RobotState</code>	<code>STARTUP</code> , <code>IDLE</code> , <code>MOVING</code> , <code>SETTLING</code> , <code>ERROR</code> , <code>RECOVERY</code> , <code>SHUTDOWN</code> .
<code>PlanResult</code>	<code>SUCCESS</code> , <code>POSE_NOT_REACHABLE</code> , <code>LINEAR_PATH_FAILED</code> .
<code>SystemStatus</code>	Aggregated status: <code>robot_state</code> , <code>plan_result</code> , <code>robot_diagnostic_flags</code> , per-joint diagnostic flags.
<code>DiagnosticFlags</code>	Bitmask flags for target/actuator saturation, jump limits, boundary safety, planner events, and hardware/tracking faults.